

Sequence Diagrams

Login

```
sequenceDiagram
    actor Staff
    participant UI as Angular Login
    participant AC as AuthenticationController
    participant AS as AuthenticationService
    participant AM as AuthenticationManager
    participant UD as CustomUserDetailsService
    participant JWT as JwtService

    Staff->>UI: username / password
    UI->>AC: POST /auth/login
    AC->>AS: login()
    AS->>AM: authenticate()
    AM->>UD: loadUserByUsername()
    UD-->>AM: BankUserDetails
    AM-->>AS: Authentication
    AS->>JWT: generateToken()
    JWT-->>AS: JWT
    AS-->>UI: LoginResponse
    UI->>UI: store token + roles
```

Open Current account with opening deposit

```
sequenceDiagram
    actor Staff
    participant Detail as CustomerDetail
    participant PolSvc as AccountPolicyService FE
    participant Dialog as OpeningDepositDialog
    participant AccFE as AccountService FE
    participant APC as AccountPolicyController
    participant AC as AccountController
    participant AS as AccountServiceImpl
    participant Pol as AccountPolicyRepository
    participant Acc as AccountRepository

    Staff->>Detail: Add Current Account
    Detail->>PolSvc: getActivePolicy(CURRENT, INR)
    PolSvc->>APC: GET /account-policies
    APC-->>PolSvc: AccountPolicyResponse
    alt openingDepositRequired
        Detail->>Dialog: open (min amount)
        Dialog-->>Detail: amount
    else not required
        Detail->>Detail: amount = 0
    end
    Detail->>AccFE: POST /accounts
    AccFE->>AC: OpenAccountRequest
    AC->>AS: openAccount()
    AS->>Pol: find active policy
    AS->>Acc: next ordinal + save
    AS->>AS: optional CREDIT + Kafka ACCOUNT_OPENED
    Acc-->>Staff: AccountResponse
```

Deposit

```
sequenceDiagram
    actor Staff
    participant UI as Deposit dialog / Accounts
    participant AC as AccountController
    participant AS as AccountServiceImpl
    participant Tx as TransactionService
    participant Pub as NotificationEventPublisher
    participant Repo as AccountRepository

    Staff->>UI: amount
    UI->>AC: POST /accounts/{id}/deposit
    AC->>AS: deposit(id, request)
    AS->>Repo: findById
    AS->>AS: validate ACTIVE + amount
    AS->>Tx: record(CREDIT)
    AS->>Repo: save (balances++)
    AS->>Pub: publish(DEPOSIT_COMPLETE)
    AS-->>UI: AccountResponse
```

Kafka notification → email

```
sequenceDiagram
    participant AS as AccountServiceImpl
    participant Pub as NotificationEventPublisher
    participant K as Kafka
    participant C as NotificationEventConsumer
    participant Mail as NotificationMailService

    AS->>Pub: publish(BankActionEvent)
    Pub->>K: bankone.notifications
    K->>C: BankActionEvent
    C->>Mail: send HTML+plain email
    Note over Mail: Mailpit local / SendGrid Render
```

Create customer with first account

```
sequenceDiagram
    actor Staff
    participant UI as Create Customer dialog
    participant CC as CustomerController
    participant CS as CustomerServiceImpl
    participant CR as CustomerRepository
    participant AS as AccountService

    Staff->>UI: customer + account fields
    UI->>CC: POST /customers
    CC->>CS: createCustomer()
    CS->>CR: save Customer
    opt accountType provided
        CS->>AS: openAccount()
    end
    CS-->>UI: Customer
```

JWT request authorization

```
sequenceDiagram
    participant UI as Angular
    participant F as JwtAuthenticationFilter
    participant JWT as JwtService
    participant UD as CustomUserDetailsService
    participant C as Controller

    UI->>F: Request + Bearer token
    F->>JWT: extractUsername / isTokenValid
    F->>UD: loadUserByUsername
    UD-->>F: BankUserDetails + roles
    F->>C: SecurityContext set
    C-->>UI: Response
```